

Benchmarking Performance Analysis of Optical Character Recognition Techniques

Muhammad Ali Naqi Hadi,
Maria Gul, Maqbool Khan*

Pak Austria Fachhochschule-
Institute of Applied Sciences and
Technology, Haripur, Pakistan
(hnaqi592, mariakhattak2015@gmail.com
maqbool.khan@scch.at

Ghadah Naif Alwakid
Department of Computer Science,
Jouf University, Skakah, 72341,
Al Jouf, Saudi Arabia
gnalwakid@ju.edu.sa

Noor Zaman Jhanjhi
Taylor's university Malaysia
NoorZaman.Jhanjhi@taylors.edu.my

Abstract—Optical Character Recognition automates the extraction of printed and handwritten text from documents; it thus is very vital in digitalizing records. This research benchmarks seven optical character recognition (OCR) engines: PaddleOCR, EasyOCR, Keras-OCR, Pytesseract OCR, OpenCV OCR, PyMuPDF OCR, and DocTR OCR, on 200 diverse CBC patient reports. The metrics taken into consideration for evaluation included execution time, accuracy, Character Error Rate, and Word Error Rate. Finally, PaddleOCR is the one that performs best: 67.28% in accuracy, 0.43 in character error rate (CER), and 0.66 in word error rate (WER). Meanwhile, Keras OCR records the highest error rates among all. The findings make an insight into picking an appropriate OCR tool, which is bound to have tremendous implications for healthcare, legal documentation, and historical preservation in guiding future OCR advancements. This research will be baseline for further research in OCR.

Keywords: OCR, text extraction, Paddle OCR, accuracy, CER, WER, CBC Reports Dataset

I. INTRODUCTION

Today and in the current generation, Optical Character Recognition (OCR) is one of the most crucial tools for converting printed or handwritten text into machine-encoded information that can be analyzed in different sectors[1]. Since historical documents and other types of records gain value and relevance with time, and they are increasingly digitized, the reliability and productivity of OCR are essential for both sciences and practical usage[2], [3]. Here, seven distinct OCR engines are compared against the CBC Reports Dataset, made from 200 patient reports from a laboratory: PaddleOCR, EasyOCR, Keras-OCR, Tesseract (in conjunction with the Python library) Pytesseract, OpenCV, PyMuPDF, DocTR. These collection of reports are semi-structured, contain handwriting style in different fonts and layout which create reasonable environment to test the OCR systems[4].

Thanks to new algorithms in machine learning and deep learning the problems related to text recognition became less sensitive and more effective[5]. While rule-based OCR systems, on the other hand, are rigid, limited to the rules and patterns it was trained with, modern OCR engines such as PaddleOCR and EasyOCR utilize deep learning for detection even on difficult layout and language that may be hard for rule-based engine to identify. Accuracy is critical in industries

including the medical sector, the law, and archive conservation since machine performance in a recognition context matters[6]. The OCR engines were assessed using the metrics of the time to execute, and the accuracy and WER as well as CER for batch sizes of [1 to 200 reports]. To evaluate each tool precision we qualitatively measured it based on the rate achieved when comparing the extracted text against a ground truth. WER and CER are used to determine the extent to which each engine in the design recognizes the input information. Therefore, design WER and CER indicate word error rate and character error rate of the engine respectively[7]. Importantly, it is seen that OCR capability differs by language, script, and the type of document being used and some of the open-source OCRs have their advantages and limitations (for example, Tesseract is considered to work efficiently with captured printed text whose contrast is high, but it lacks the ability to correctly recognize handwritten text or faded content on paper with low resolution)[8]. As pointed out by Patel et al, (2023), the modern deep learning-based OCR algorithms are usually superior to the previous approach, but they demand large datasets and computational power for its training. This paper reviews multiple OCR tools and discusses the advantages as well as drawbacks of each option in order to allow users choose the best match for their task. The subsequent sections provide an account of the method followed and discusses the findings obtained and concludes with a recommendation of the OCR tool suitable for an application and the future work guidelines to optimise the OCR technology[9].

II. MAIN CONTRIBUTIONS OF THE PAPER

This paper systematically examines seven widely used OCR engines: PaddleOCR, EasyOCR, Keras-OCR, Pytesseract, OpenCV, PyMuPDF, and DocTR. The evaluation is based on the CBC Reports Dataset, comprising 200 patient reports with varied fonts, layouts, and handwriting styles. Each OCR engine's performance is assessed through two key metrics: execution time and accuracy. Accuracy is evaluated at two levels word and character using Word Error Rate (WER) and Character Error Rate (CER) to provide a detailed measure of precision. Execution times are recorded for different batch sizes (1, 50, 100, 150, and 200 reports), allowing for a compre-

hensive analysis of scalability and processing speed for each engine. The study rigorously compares each OCR engine's output against a predefined ground truth to ensure an accurate evaluation. Among the engines tested, PaddleOCR is identified as the top performer, achieving 67.28% accuracy, a CER of 0.43, and a WER of 0.66, making it highly recommended for applications requiring high precision. Additionally, the paper highlights certain limitations of the evaluated OCR tools. For instance, Keras OCR tends to have a higher error rate, and Tesseract performs poorly with low-resolution or handwritten text.

III. RELATED WORK

The growth and performance evaluation of Optical Character Recognition technology have been prominent in research in connection with the recognition accuracies, speeds, and flexibilities by using different languages, various types of scripts, and characters or features of documents. A dynamic process of OCR translates or transforms type-written and or hand-written forms into a dynamic text, active and involved in the process of digitizing. Tesseract is one of the popular OCR engines[10]. Its performance is 93% on high-quality modern print output in Gujarati and 90% in Hindi. It however fails to work well with old corrupted Does not work well with documents, especially when the text is difficult. A comparative study of other OCR tools such as Google Docs OCR, Online OCR, and Free OCR shows that Google Docs OCR has an excellent accuracy in processing high-resolution images, while Free OCR tool performs differently depending on the quality and format of the image[11]. It can be concluded in the clear OCR that the deep learning models, CNN and LSTM, have managed to yield much in the complex text, especially through considering complexity as well as Big improvement, whereas the said models do demand large databases and preprocessing to obtain a good performance[12]. For complicated designs and multiple languages, OCR optimization methods have emerged with promising strategies in combining different results from various OCR engines towards improvement in accuracy through the detection and correction of all results[13]. Methodological studies of binarization and multi-resolution techniques, like Otsu's method, have established efficient methods for generating inaccurate and historical documents, and Otsu's method has appeared in tests of twelve versions of commercial OCR volumes, including Omnipage and Corel[14].

Additionally, simple models for OCR like PP-OCR, designed for several applications from office automation to online training-can be used in numerous global conditions since this minimizes the model size and changes the number of groups, truth. Parallel processing in the OCR task-based and region-specific pipelines was applied[15]. It investigates effectiveness in speeding up tasks, such as document classification, barcode recognition and MRI images classification [16], [17], [18]. Methods of meta-algorithms that are resource hungry but result in high accuracy are utilized. This is because it applies voting and the amalgamation of various algorithm outputs to increase the accuracy rate, thus becoming more practical for

applications demanding high levels of accuracy[19]. Research that combines neural networks, support vector machines, and pattern matching demonstrates that pre-processing and feature extraction could significantly enhance the recognition accuracy in healthcare and financial applications. Devanagari and Gurmukhi Script OCR, Research also indicates that classifiers using gradient and linear features, particularly with the use of a support vector machine, often achieve an excellent accuracy over 95%[20]. Gradient-based features are proved to outperform Gabor features in Chinese character recognition, especially for complicated handwriting, making gradient extraction technology the most reliable in the field. In natural image OCR, comparing Tesseract, ABBYY FineReader, and GOCR, it would appear that Tesseract and ABBYY FineReader work well with high-contrast and structured images; GOCR is not able to keep up with the level of complexity in natural images and does not work very well with scenes that lack adequate settings[21]. More research on Tesseract and Transym OCR. It was because of its superior performance with grayscale images and the type of document or image will be forming one of the prime criterias in determining the OCR, such that, very advanced OCR evaluation measures, such as the ZoneMapAltCnt and also the XML-to-XML evaluation of the segmentation, as well as complexities from content recognition[22]. These new Euclidean distance metrics-based neural network methodologies have been developed to enhance the state-of-the-art OCR accuracy based on the mapping of calculated distance metrics of character patterns[23]. In addition, methods of feature extraction such as binary and vector representations play essential roles in the building of OCR systems that are strong, flexible, and tolerant toward shape variations and resolutions that can be associated with a wide range of character types. Applications in medical contexts further indicate the potential of hybrid CNN-based OCR models, highly effective at recognizing handwritten text, despite challenges inherent to the medium. The literature depicts that the performance of OCR depends on the variables such as image quality, document type, script complexity, and algorithmic approach. The traditional methods, such as Tesseract, are effective in structured documents, but deep learning and hybrid models reveal a good potential for the complex, high-dimensional character recognition tasks and therefore further development in OCR research toward even more accurate, adaptable, and application-specific systems.

IV. METHODOLOGY

A. Data Preparation

1) *Dataset*: The CBC Reports Dataset is a test including 200 laboratory-acquired patient reports.

2) *Report Processing*: Take a single report initially to measure time at the beginning of the process, and then continue to 50, 100, and 150 and finally the total number of 200 reports. This way, time efficacy can be gauged.

B. OCR Application

1) *OCR Engines*: Query all seven OCR engines individually through Python-based libraries PaddleOCR, EasyOCR, Keras-OCR, Pytesseract, and OpenCV for terminologies detection as well as catching end-to-end automation processes.

2) *Execution*: There are several OCR engines which will process the dataset individually by taking text data from each report.

C. Time Measurement

1) *Execution Time Calculation*: The time taken by each OCR engine is recorded and then used for processing as for different batch sizes (single, 50, 100, 150, and 200 reports). This helps in analyzing the performance scalability of each engine.

D. Accuracy Assessment

1) *Ground Truth Comparison*: Individual evaluation of each OCR output compared to a predefined ground truth (as Microsoft Word.docx file) as the reference.

2) *Libraries to Compare*: The libraries python-docx, for managing the components of a document; and its Python counterpart diff lib, used to calculate similarity ratios and highlight OCR output deviations.

E. Error Metrics

1) *Character Error Rate (CER)*: This metric counts any deletions, insertions, or substitutions at the character level, enabling evaluation based on character-wise accuracy. CER works well in spotting symbol, typographical, and abbreviation mistakes.

2) *Word Error Rate (WER)*: WER provides a performance measure focused on complete words rather than individual phonemes or distinctions, offering insights into recognition accuracy at the word level.

F. Report Generation

1) *Result Aggregation*: Summarize results in terms of both execution time and accuracy for each OCR engine.

2) *Output Format*: The final report will categorize performance data by batch sizes and accuracy rates (CER and WER), enabling clear comparison across engines.

V. RESULTS AND DISCUSSION

The following sections compare the performance of different OCR methods (PaddleOCR, EasyOCR, Keras-OCR, Pytesseract, OpenCV, PyMuPDF, and docTR) across processing time, accuracy, and error metrics. The analysis is based on varying report sizes to illustrate each OCR method's strengths and weaknesses.

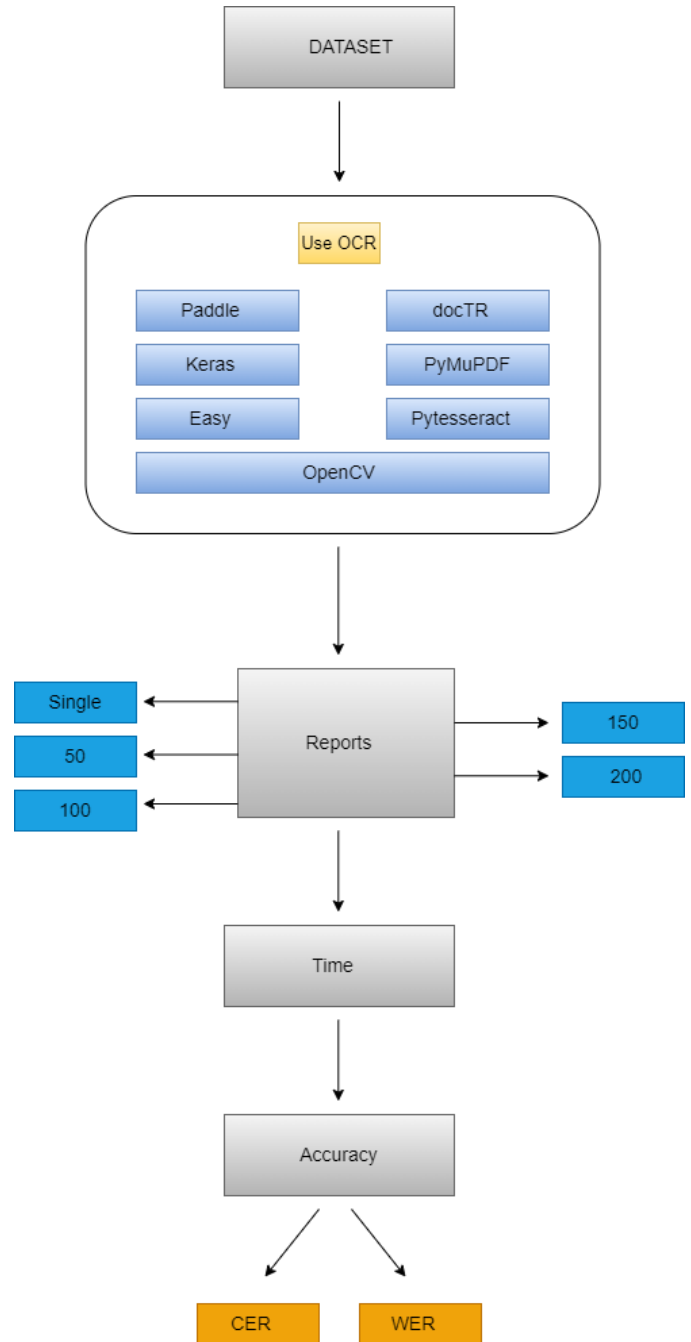


Fig. 1: OCR Model Evaluation Workflow for Accuracy and Performance Metrics

A. Processing Time Analysis

Comprehensive comparison of different types of OCR algorithms demonstrates the contribution of certain procedures and identifies critical issues concerning documents' processing rate when increasing the flow of documents. As shown in Fig. 2, no matter across which dataset, PaddleOCR performs uniformly better than the counterparts it is faster and can handle large amounts of data efficiently which is particularly essential in applications like healthcare that need fast data extraction.

Despite the fact that EasyOCR consumes little more time than PaddleOCR does, for larger report sizes it offers equally impressive performance due to the deep training models that ensure that the accuracy and speed of the OCR system are well balanced. On the other hand, the longest processing times are affirmed by OpenCV, hence indicating its scalability challenges more so for perhaps complicated layouts or even large scale digitization of documents. Keras-OCR, Pytesseract, PyMuPDF and docTR show moderate throughputs with performance times that are closer to PaddleOCR/EasyOCR but less productive compared to OpenCV; Keras-OCR and Pytesseract are not efficient for massive tasks because of computational load while PyMuPDF and docTR are still relatively primitive in their maturity. In general, we have seen that the PaddleOCR has an edge in terms of speed and the degree of scaling which is the reason why it is popular to be used for massive scale data processing for OCR.

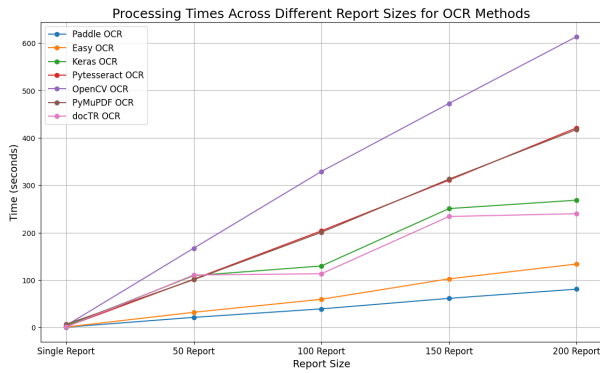


Fig. 2: Processing Time Comparison for Different Report Sizes

B. Distribution of Processing Times

As the Fig. 3, also shows above the box plot reveals how processing times of different OCR approaches change with more extensive data. The two cartoons are the average processing time and the quartile chart with two whiskers illustrating the time range of each method in relation to Q1 and Q3 of the data size to enhance the readability of the study. The average response time and the standard deviation of PaddleOCR and EasyOCR are both comparatively low, thus, indicating their ability to deal with large amounts of data. It is also worthy of notice that PaddleOCR demonstrates stable work with minimum variability even with high load which is necessary for rapid large-scale tasks. Instead, OpenCV, PyMuPDF, and docTR see the increase in the average times and the divergence in the range of the processing time of the shortest and longest duration when the amount of data increases indicating the issues with managing the heavier workload and complex structure of the document layout. For those projects that require an immediate and high-throughput processing, PaddleOCR and EasyOCR are ideal, while for those projects that do not require urgent processing but are large-scale, OpenCV, PyMuPDF and docTR can be the best solution for the job. Finally, this chart is helpful in decisions

about which OCR tool to apply in terms of data size or when deadline is a concern for the task and its completion.

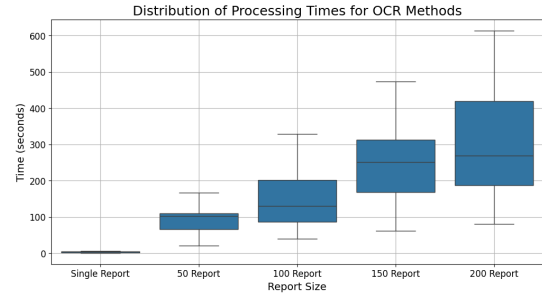


Fig. 3: Distribution of Processing Times for Different OCR Methods

C. Processing Time Histogram

The histogram in the picture we are talking about shows how long it takes for different OCR (Optical Character Recognition) methods to do their thing. It's like a graph that lets you see at a glance which ones are the speed demons and which ones are more like the tortoises. The x-axis is like a timeline in seconds, and the y-axis tells you how often each engine does its job in those time slots. So, PaddleOCR and EasyOCR are the cool kids on the block, finishing their stuff mostly in the 0-50 seconds range, which is pretty zippy. That means they are great for when you have got a ton of data to chew through quickly and consistently. They are like the Usain Bolts of OCR, if you catch my drift. But then there is Pytesseract and OpenCV, who are more like marathon runners. They can get the job done, sure, but they take their sweet time with the larger datasets, stretching up to 400 seconds for Pytesseract and 600 seconds for OpenCV. That is probably because they are using some old-school OCR techniques that do not play so well with big data.

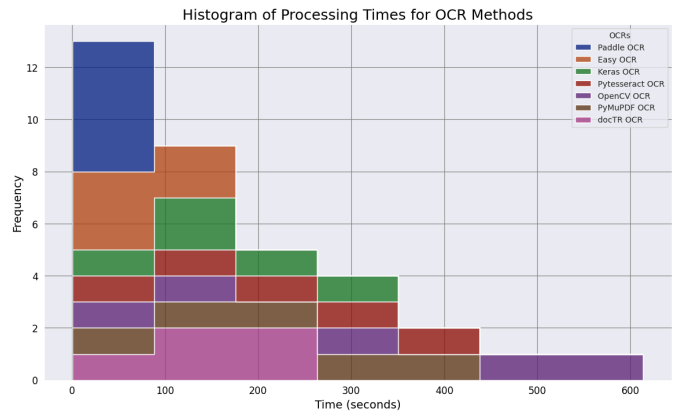


Fig. 4: Histogram of Processing Times for Different OCR Methods

D. Processing Time Comparison Table

TABLE II shows how long it takes for each OCR method to get stuff done with reports of different sizes. PaddleOCR is like the speed demon here, zipping through them faster

than you can say "Optical Character Recognition." It is crazy efficient, even when dealing with a bunch of reports at once. For example, it takes less than a second to handle one report, and just over a minute to tackle 200 of them. That is some serious horsepower! On the flip side, OpenCV is kind of like the tortoise in this race. It is not terrible with small reports, but when you throw a bunch of them at it, it starts to chug. It takes more than four seconds for just one and over ten minutes for 200 reports. That is a pretty big jump, and not so great when you have got a mountain of paperwork to digitize. EasyOCR and PyMuPDF are the middle-of-the-road folks. They are not as fast as PaddleOCR, but they are not as sluggish as OpenCV with larger reports either. They handle medium-sized stacks of reports without breaking a sweat, but they are not built for the big leagues like PaddleOCR. So if you have got a decent chunk of reports to process, PaddleOCR's your best buddy. It keeps its cool no matter how many reports you throw its way.

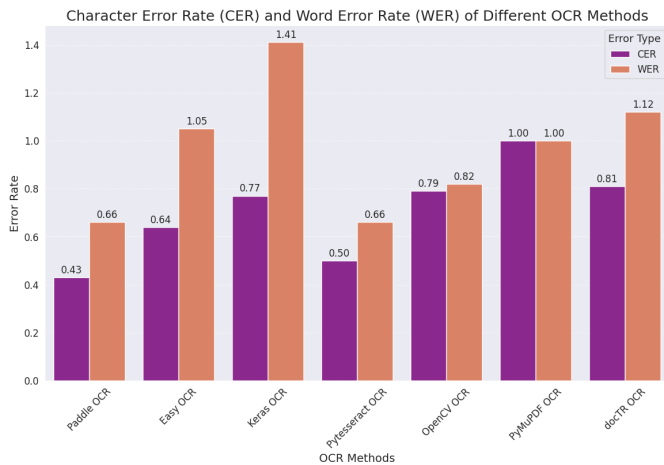


Fig. 5: CER and WER for Different OCR Methods

TABLE I: OCR Performance Metrics

OCRs	Accuracy (%)	CER	WER
Paddle OCR	67.28%	0.43	0.66
Easy OCR	27.24%	0.64	1.05
Keras OCR	3.51%	0.77	1.41
Pytesseract OCR	14.63%	0.50	0.66
OpenCV OCR	14.63%	0.79	0.82
PyMuPDF OCR	6.95%	1.00	1.02
docTR OCR	17.21%	0.81	1.12

E. Accuracy Comparison

Fig. 6 shows the accuracy for each OCR method, with PaddleOCR achieving the highest at 67.28%. This is followed by EasyOCR at 27.24%. Keras-OCR has the lowest accuracy at 3.51%, making it less suitable for precise text extraction tasks.

F. Error Rates (CER and WER)

Fig. 5 shows a bar graph depicting the CER and WER of each OCR engine used for character and word recognition. As for the engines, PaddleOCR has the best performance with

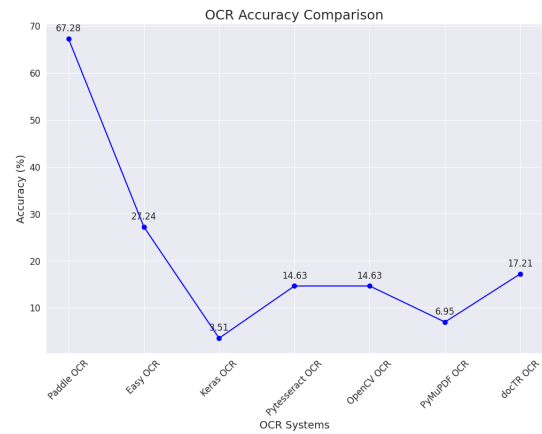


Fig. 6: Accuracy of Different OCR Methods

CER: 0.43 and WER: 0.66. It is really good at reading stuff, so you can trust it for stuff that needs to be perfect. Then there is Keras-OCR, which is not doing so hot with more mistakes - 0.77 for characters and 1.41 for words. It is not so great with all the fancy layouts and stuff, so maybe do not use it when you need everything to be just right. EasyOCR, Pytesseract, and docTR, they are okay. They are not as amazing as PaddleOCR, but they are not terrible either. They make a few more mistakes, but for basic stuff, they will do the job.

VI. CONCLUSION

This work tested seven OCR tools; PaddleOCR, EasyOCR, Keras-OCR, Pytesseract, OpenCV, PyMuPDF, and docTR on 200 samples of CBC reports and metrics like time taken, accuracy, CER, and WER were measured. PaddleOCR was found to be most effective and accurate, with 67.28% accuracy, 0.43 CER and 0.66 WER and therefore suited best for those applications where real-time results are needed. EasyOCR has also passed this test but, it was not as accurate as the first one and took more time to process images.

On the other hand, Keras-OCR faced more errors and is not the best for high accuracy applications. While using OpenCV and Pytesseract, I witnessed a moderate performance, but text recognition in low-resolution images and layouts with much complexity took more significant time as compared to high-performance OCR such as PaddleOCR and EasyOCR. In lands of OCR, these outcomes ensure that the performance is inconsistent; engine selection also greatly determines the project's duration and overall accuracy foundation particularly for commercial applications like digitizing health records, document archiving, or legal services.

Such applications include scenarios where accuracy and speed are of essence by adopting the PaddleOCR. The research also highlights the need to improve on the engine's error prone rate and increase its speed for other OCR engines such as Keras-OCR and Pytesseract. In a similar manner, as advances in OCR continue to emerge these findings serve to provide developers and professionals with a clear roadmap of how and where to implement OCR as well as the best tools to use

TABLE II: OCR Processing Times for Different Numbers of Reports (in seconds)

OCR Engine	Single Report	50 Reports	100 Reports	150 Reports	200 Reports
PaddleOCR	0.54 s	21.20 s	39.08 s	61.32 s	80.72 s
EasyOCR	0.81 s	31.93 s	59.30 s	102.44 s	133.71 s
docTR OCR	1.01 s	110.57 s	113.23 s	234.04 s	239.85 s
Keras OCR	3.53 s	109.25 s	129.59 s	250.72 s	268.43 s
PyMuPDF OCR	6.62 s	100.69 s	200.59 s	312.78 s	417.50 s
Pytesseract OCR	1.62 s	101.73 s	203.69 s	310.75 s	421.08 s
OpenCV OCR	4.07 s	167.15 s	329.03 s	472.44 s	613.56 s

to enhance the overall speed and accuracy of text recognition across industries.

REFERENCES

- [1] S. Guan and D. Greene, "Advancing post-ocr correction: A comparative study of synthetic data," *arXiv preprint arXiv:2408.02253*, 2024.
- [2] A. Shetty and S. Sharma, "Ensemble deep learning model for optical character recognition," *Multimedia Tools and Applications*, vol. 83, no. 4, pp. 11 411–11 431, 2024.
- [3] M. A. Khan, F. Ahmad, K. Khan, and M. Khan, "Pashto language handwritten numeral classification using convolutional neural networks," in *International Conference on Forthcoming Networks and Sustainability in the AIoT Era*. Springer, 2024, pp. 287–297.
- [4] M. Singh, P. Gayathri, and P. Kandukuri, "Ocr for devanagari script," in *MATEC Web of Conferences*, vol. 392. EDP Sciences, 2024, p. 01128.
- [5] T. Malathi, D. Selvamuthukumaran, C. D. Chandar, V. Niranjana, and A. Swashtika, "An experimental performance analysis on robotics process automation (rpa) with open source ocr engines: Microsoft ocr and google tesseract ocr," in *IOP Conference Series: Materials Science and Engineering*, vol. 1059, no. 1. IOP Publishing, 2021, p. 012004.
- [6] P. Batra, N. Phalnikar, D. Kurmi, J. Tembhurne, P. Sahare, and T. Diwan, "Ocr-mrd: performance analysis of different optical character recognition engines for medical report digitization," *International Journal of Information Technology*, vol. 16, no. 1, pp. 447–455, 2024.
- [7] M. Bardvall and I. Hassle, "Automating invoice recognition: A comparative study of large language models and ocr/ml technologies," 2024.
- [8] K. Joshi and H. Arolkar, "Comparative analysis of outcomes of tesseract ocr for different languages," in *2024 5th International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV)*. IEEE, 2024, pp. 95–100.
- [9] Y. Li, "Synergizing optical character recognition: A comparative analysis and integration of tesseract, keras, paddle, and azure ocr," Ph.D. dissertation, University of Sydney, 2024.
- [10] S. K. Manocha and P. Tewari, "Comparative study of deep learning models for devanagari ocr," in *2021 International Conference on Smart Generation Computing, Communication and Networking (SMART GEN-CON)*. IEEE, 2021, pp. 1–7.
- [11] Salma, M. Saeed, R. ur Rahim, M. Gufran Khan, A. Zulfiqar, and M. T. Bhatti, "Development of anpr framework for pakistani vehicle number plates using object detection and ocr," *Complexity*, vol. 2021, no. 1, p. 5597337, 2021.
- [12] J. Smith *et al.*, "Ocr performance prediction using cross-ocr alignment," *Journal of ...*, 2022.
- [13] P. Jain, K. Taneja, and H. Taneja, "Which ocr toolset is good and why: A comparative study," *Kuwait Journal of Science*, vol. 48, no. 2, 2021.
- [14] M. R. Gupta, N. P. Jacobson, and E. K. Garcia, "Ocr binarization and image pre-processing for searching historical documents," *Pattern Recognition*, vol. 40, no. 2, pp. 389–397, 2007.
- [15] D. Vedhaviyash, R. Sudhan, G. Saranya, M. Safa, and D. Arun, "Comparative analysis of easyocr and tesseractocr for automatic license plate recognition using deep learning algorithm," in *2022 6th International Conference on Electronics, Communication and Aerospace Technology*. IEEE, 2022, pp. 966–971.
- [16] M. F. Almufareh, M. Imran, A. Khan, M. Humayun, and M. Asim, "Automated brain tumor segmentation and classification in mri using yolo-based deep learning," *IEEE Access*, vol. 12, pp. 16 189–16 207, 2024.
- [17] M. F. Almufareh, S. Kausar, M. Humayun, and S. Tehsin, "Empirical assessment of transfer learning strategies for dementia classification using mri images," *Journal of Disability Research*, vol. 3, no. 4, p. 20240047, 2024.
- [18] S. Saeed, H. B. Haroon, N. Z. Jhanjhi, M. Naqvi, and M. Ahmad, "A systematic mapping study of low-grade tumor of brain cancer and csf fluid detecting in mri images through multi-algorithm techniques," *Approaches and Applications of Deep Learning in Virtual Medical Care*, pp. 168–201, 2022.
- [19] C. A. L. Montesinos and E. J. E. Cárdenas, "Verification of peruvian identity document fraud through ocr, hash algorithm, and simulated blockchain database," in *Knowledge Management and Artificial Intelligence for Growth: Cases from Emerging and Developed Economies*. Springer, 2024, pp. 165–188.
- [20] S. Vijayarani and A. Sakila, "Performance comparison of ocr tools," *International Journal of UbiComp*, 2015.
- [21] C. H. Yu, P. C. Y. Yau, Q. Cao, C. K. Seow, P. Pang, and D. Wong, "Machine learning-enhanced automation for invoice reconciliation: Ocr-based solutions for accuracy and efficiency in logistics industry," in *Intelligent Systems Conference*. Springer, 2024, pp. 325–336.
- [22] S. J. Simske, "Parallel processing considerations for image recognition tasks," in *Parallel Processing for Imaging Applications*, vol. 7872. SPIE, 2011, pp. 59–73.
- [23] M. D. Alahmadi and M. Alshangiti, "Optimizing ocr performance for programming videos: The role of image super-resolution and large language models," *Mathematics*, vol. 12, no. 7, p. 1036, 2024.